Case Study 27: Advanced Numeric Pattern Generator

1. Problem Statement

Produce a symmetric numerical diamond or hollow numerical pattern based on user-specified dimensions, incorporating robust validation for invalid or even dimensions and delivering clean, aligned output.

2. Objectives

- Validate dimension inputs to ensure positive odd integers for perfect symmetry.
- Construct top and bottom symmetric numeric sequences using nested loops.
- Format spacing and number sequences for precise visual alignment.

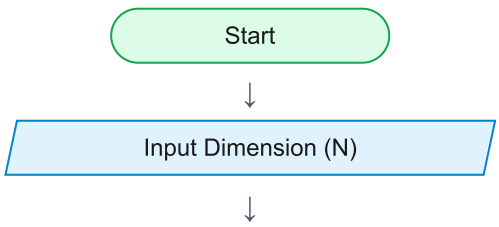
3. Assumptions

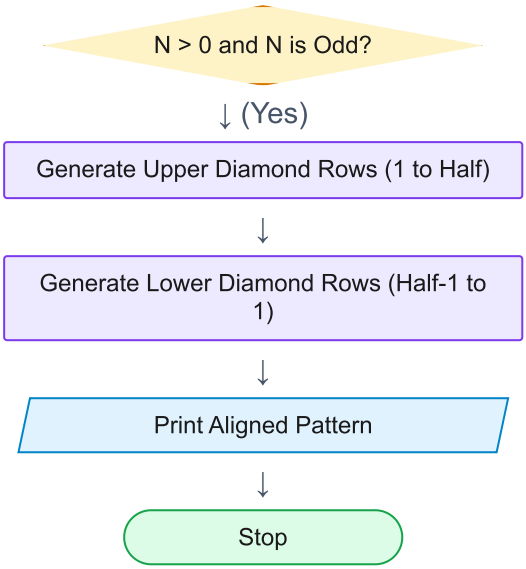
- The pattern height `N` must be an odd positive integer (e.g., 3, 5, 7, 9).
- If an even integer is entered, the program rejects it or increments it to the next odd integer.
- Numbers increment from 1 up to the peak row value and decrement symmetrically.

4. Algorithm

1. Start.
2. Read pattern dimension `N`.
3. Validate `N`: If `N <= 0` or `N % 2 == 0`, display an error message and terminate or adjust input.
4. Calculate `half = N // 2 + 1` (number of rows in the upper diamond section).
5. **Upper Half (Rows 1 to half):**
   - Print `half - i` leading spaces.
   - Print ascending numbers from 1 to `i`.
   - Print descending numbers from `i - 1` down to 1.
6. **Lower Half (Rows half - 1 down to 1):**
   - Print `half - i` leading spaces.
   - Print ascending numbers from 1 to `i`.
   - Print descending numbers from `i - 1` down to 1.
7. Stop.

5. Flowchart





6. Pseudocode

```
START INPUT N IF N <= 0 OR N % 2 == 0 THEN PRINT "Error: Dimension must be a positive odd integer." EXIT ENDIF half = (N // 2) + 1 // Upper Pyramid FOR i FROM 1 TO half DO PRINT (half - i) spaces FOR j FROM 1 TO i DO PRINT j ENDFOR FOR j FROM i-1 DOWNTO 1 DO PRINT j ENDFOR PRINT newline ENDFOR // Lower Pyramid FOR i FROM half-1 DOWNTO 1 DO PRINT (half - i) spaces FOR j FROM 1 TO i DO PRINT j ENDFOR FOR j FROM i-1 DOWNTO 1 DO PRINT j ENDFOR PRINT newline ENDFOR STOP
```

7. Complete Python Source Code

```
# Advanced Numeric Pattern Generator def generate_numeric_diamond(n): # Input Validation if n <= 0: print("Error: Dimension must be greater than zero.") return if n % 2 == 0: print(f"Warning: {n} is even. Converting to odd dimension {n + 1} for symmetry.\n") n += 1 half = (n // 2) + 1 print("=" * 45) print(f" NUMERIC DIAMOND PATTERN (N = {n})") print("=" * 45) # Upper section (including middle row) for i in range(1, half + 1): # Print leading spaces for s in range(half - i): print(" ", end="") # Print ascending numbers for j in range(1, i + 1): print(j, end="") # Print descending numbers for j in range(i - 1, 0, -1): print(j, end="") print() # Lower section for i in range(half - 1, 0, -1): # Print leading spaces for s in range(half - i): print(" ", end="") # Print ascending numbers for j in range(1, i + 1): print(j, end="") # Print descending numbers for j in range(i - 1, 0, -1): print(j, end="") print() print("=" * 45) # Test Runs generate_numeric_diamond(5) print() generate_numeric_diamond(4) # Tests even-number validation
```

8. Test Case and Expected Output

Test Case 1 (Valid Input: N = 5):

```
     ========================================== NUMERIC DIAMOND PATTERN (N = 5)
========================================== 1 121 12321 121 1
==========================================
```

Test Case 2 (Even Input Handled: N = 4):

```
     Warning: 4 is even. Converting to odd dimension 5 for symmetry.
========================================== NUMERIC DIAMOND PATTERN (N = 5)
========================================== 1 121 12321 121 1
==========================================
```

9. Conclusion

The Advanced Numeric Pattern Generator successfully validates dimension inputs, handles even-dimension edge cases automatically, and utilizes nested loops to generate perfectly centered, symmetric numerical diamond outputs.